# Sequence Data and Embeddings

Advanced Machine Learning

Sam Ogden

# DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

# Learning objectives

- Explain why sequences differ from independent examples: order matters and length varies
- Name sequences that are not text, and say whether their items are numbers or symbols
- Turn text into tokens and ids, and say what each choice gains and loses
- Encode text as one-hot and bag-of-words vectors, and say what each cannot represent
- Describe an embedding as a learned vector per symbol, and explain how word2vec learns one from neighbours

# Sequences

Order is data

# Sequences are everywhere



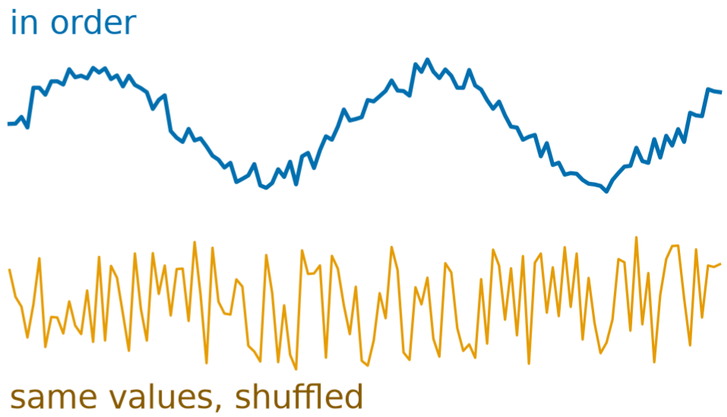

- A **sequence** is a list of items where the position of each one matters
- The items can be words, measurements, letters in a gene, pages a user visited, notes
- This lecture is mostly about words, because that is where the ideas are easiest to see. The same ideas apply to all of these

# Order matters

the cat sat on the mat and the cat ran

↓ same items, shuffled

cat the the the cat sat mat ran and on




- The same items in a different order can mean something different, or nothing at all
- Until now an example was a row of features or an image that we could treat on its own. In a sequence, **where** an item sits is part of the example
- Sequences also have different lengths: a sentence of 10 words, a sentence of 40

# Two kinds of items

## Numbers

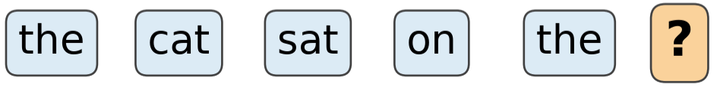
- Sensor readings, prices, audio samples
- **Already numbers**, so they can go into a model directly
- The order is the part we must keep

## Symbols

- Words, letters in DNA, pages, notes
- A choice from a list, **not a number**
- We must first decide how to turn each one into numbers

The rest of this lecture is about symbols, and text is our main example

# Predict the next item





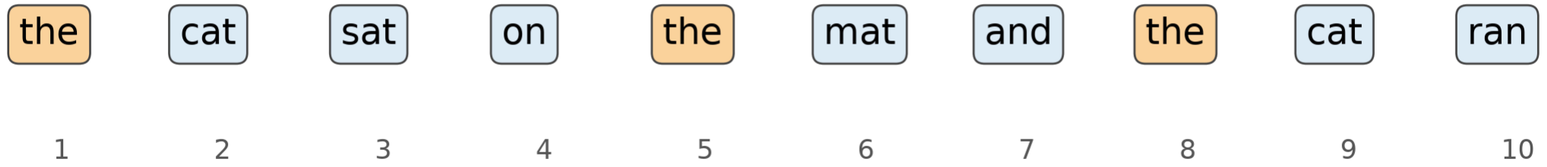


- The simplest sequence task: given the items so far, predict the **next** one
- It fits every example so far: the next word, the next sensor reading, the next note, the next page
- No separate labels are needed. The next item in the data is the answer

# The Big Idea

**Predict each item from the ones before it, using the same model at every position.**

# Our running sentence




- 10 items, 7 different words. We trace this one sentence by hand through the whole lecture
- “the” appears 3 times and is followed by cat, mat, cat. To predict what follows it, the words before it matter

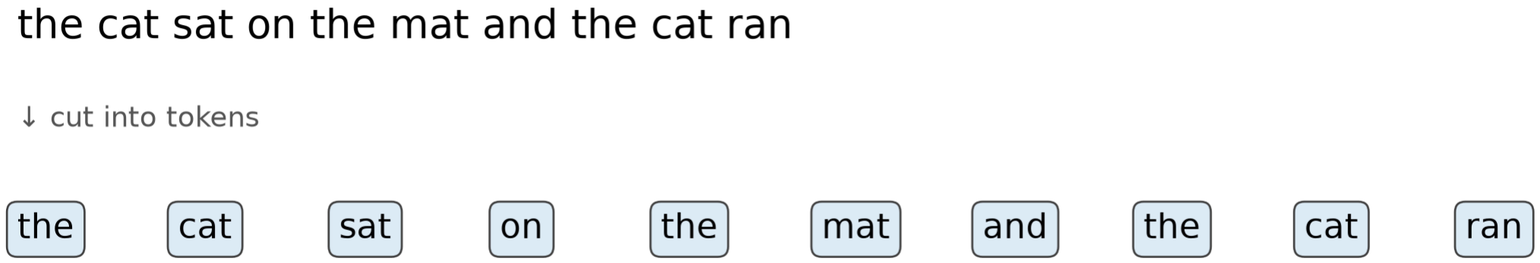
# Takeaway

**A sequence is data where order matters, and the basic task is to predict the next item from the ones before it.**

# Text to numbers

A model only sees numbers

# From text to tokens




- A **token** is the unit we choose to cut a sequence into
- For text, splitting on spaces is the simplest choice. It is a choice, not the only one
- Symbols in other sequences need the same step: a letter of DNA, or a page visited, is a token

# Vocabulary and ids

| Token | the | cat | sat | on | mat | and | ran |
|---|---|---|---|---|---|---|---|
| Id | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
| Count | 3 | 2 | 1 | 1 | 1 | 1 | 1 |

- The **vocabulary** is the list of distinct tokens, here 7. Each gets an **id**, its position in the list
- The sentence becomes a list of ids: [0, 1, 2, 3, 0, 4, 5, 0, 1, 6]. An id is only a label: id 1 is not “more” than id 2

# Special tokens

| Token | What it is for |
|---|---|
| `<unk>` | Not in the vocabulary: rare or new words |
| `<pad>` | Filler, so a batch has one length |
| `<bos>`, `<eos>` | Start and end of a sequence |

- The vocabulary is fixed from the training text. Shorter sequences are padded to share a batch
- Names vary (`<s>` and `</s>`, `[CLS]` and `[SEP]`), the roles do not
- From here on our sentence is wrapped: `<bos>` the cat sat on the mat and the cat ran `<eos>`, 12 tokens

# What a choice loses

| Choice | Before | After | What is lost |
|---|---|---|---|
| Lowercase everything | `The cat` and `the cat` | `the cat` | Which word began a sentence |
| Drop punctuation | `Stop. Go!` | `stop go` | Where sentences end |
| Keep only common words | `a quokka sat` | `a <unk> sat` | The rare word itself |

- Every preprocessing step throws something away. Whether that matters depends on the task
- Lowercasing is fine for sorting reviews by topic, and not fine for finding names

# Characters, words, or pieces of words?

| Token | Vocabulary size | Sequence length for our sentence |
|---|---|---|
| Characters | 12 | 38 |
| Words | 7 | 10 |
| Pieces of words | in between | in between |

- Characters: a tiny vocabulary, but long sequences. Words: short sequences, but a huge vocabulary and new words keep arriving
- Pieces of words (subwords) split a rare word into common parts, such as `un` + `happi` + `ness`. Real systems use these
- The same tradeoff holds for DNA: single letters, or groups of letters called k-mers

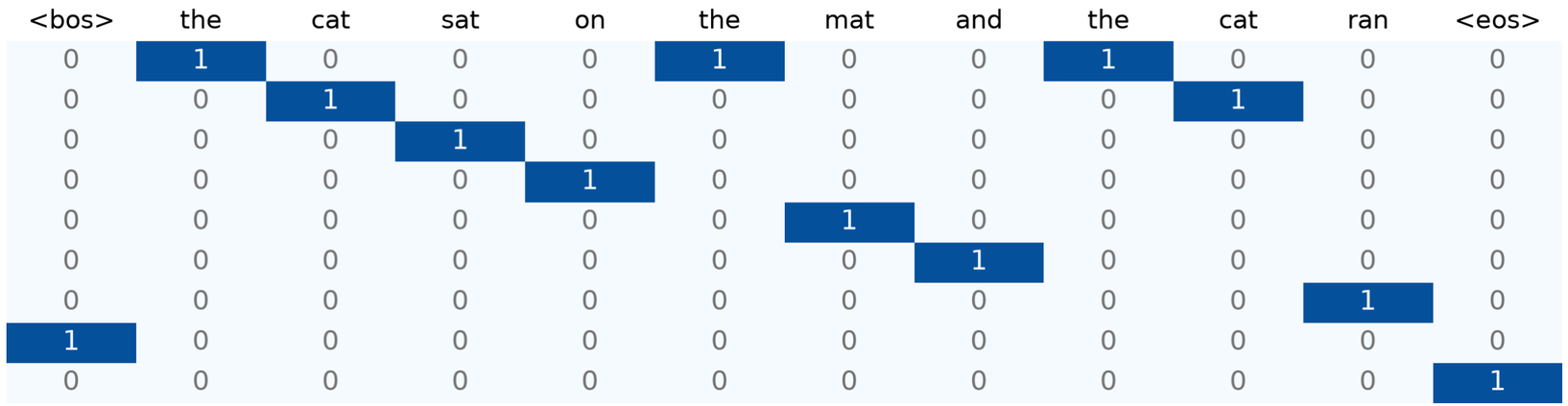
# Takeaway

**Turning a sequence into tokens decides what the model can see and how long its sequences are.**

# Encoding words

Simple ways to turn tokens into vectors

# One-hot vectors




- A token is one of a fixed list, a category, just like a class label. So we do what we did for class labels: a **one-hot** vector, with a 1 in the slot of the word and 0 elsewhere. Each column is one token of the sentence
- Distance between “cat” and “mat”: 1.41. Between “cat” and “the”: 1.41. Every pair of different words is the same distance apart

# Bag of words

| Text | the | cat | sat | on | mat | and | ran |
|---|---|---|---|---|---|---|---|
| the cat sat on the mat | 2 | 1 | 1 | 1 | 1 | 0 | 0 |
| the mat sat on the cat | 2 | 1 | 1 | 1 | 1 | 0 | 0 |
| the cat ran | 1 | 1 | 0 | 0 | 0 | 0 | 1 |

- A **bag of words** encodes a whole text as one vector of counts, one slot per vocabulary word
- Any length of text gives a vector of the same size
- The first two texts get the **same** vector. The order is gone
- Good for simple jobs such as sorting by topic. A common tweak, TF-IDF, down-weights words that appear in every text

# What these cannot say

## No sense of similarity

- Every pair of different words is equally far apart
- “cat” is no closer to “mat” than to “the”, and no closer to a word like “kitten”
- A model gets no help from words that behave alike

## No sense of context

- One-hot: a word is the same vector wherever it appears
- Bag of words: the order of the words is thrown away
- “the cat sat on the mat” and “the mat sat on the cat” look the same

Both are fine for some jobs, and neither gives the vectors we want: ones where closeness means something

# Embeddings

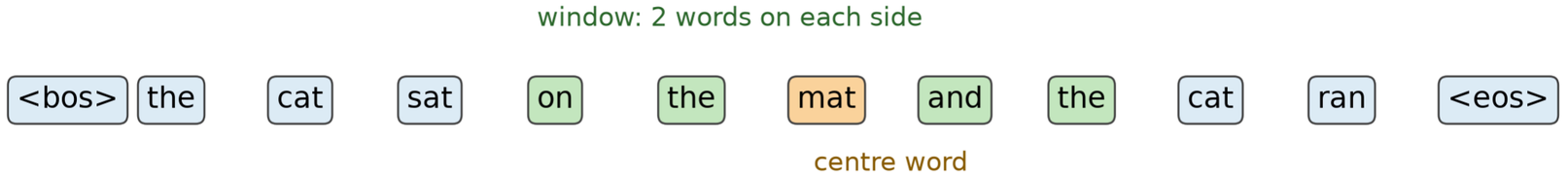
Learn the vectors

# The Big Idea

> **“You shall know a word by the company it keeps.”**

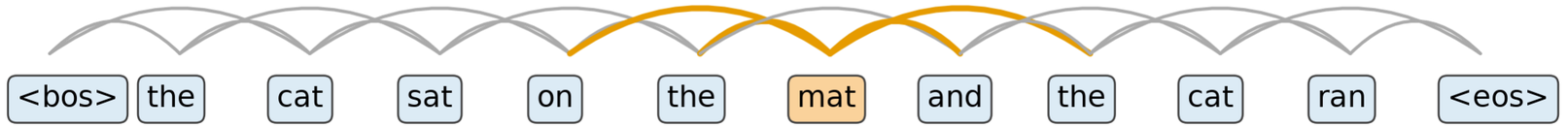
- J. R. Firth, 1957
- A word’s meaning shows in the words that appear around it. That is the idea behind **word2vec**

# word2vec: predict the neighbours




- Slide a window along the text. The word in the middle is the **centre word**, the words around it are its **neighbours**
- **Skip-gram**: given the centre word, predict each of its neighbours. The vectors learned along the way are the embeddings

# Skip-gram pairs




- Slide the window so every word takes a turn as the centre. Each arc links two words that share a window, giving two pairs: one in each direction
- Orange: the window we just saw. “mat” as the centre gives mat → on, mat → the, mat → and, mat → the
- 42 pairs in all. The labels are free, like the same/different pairs last lecture

# A word, in terms of its neighbours

| | the | cat | sat | on | mat | and | ran | <bos> | <eos> |
|---|---|---|---|---|---|---|---|---|---|
| the | 0 | 2 | 2 | 1 | 2 | 2 | 1 | 1 | 0 |
| cat | 2 | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| sat | 2 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| on | 1 | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 0 |
| mat | 2 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| and | 2 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| ran | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| <bos> | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| <eos> | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |

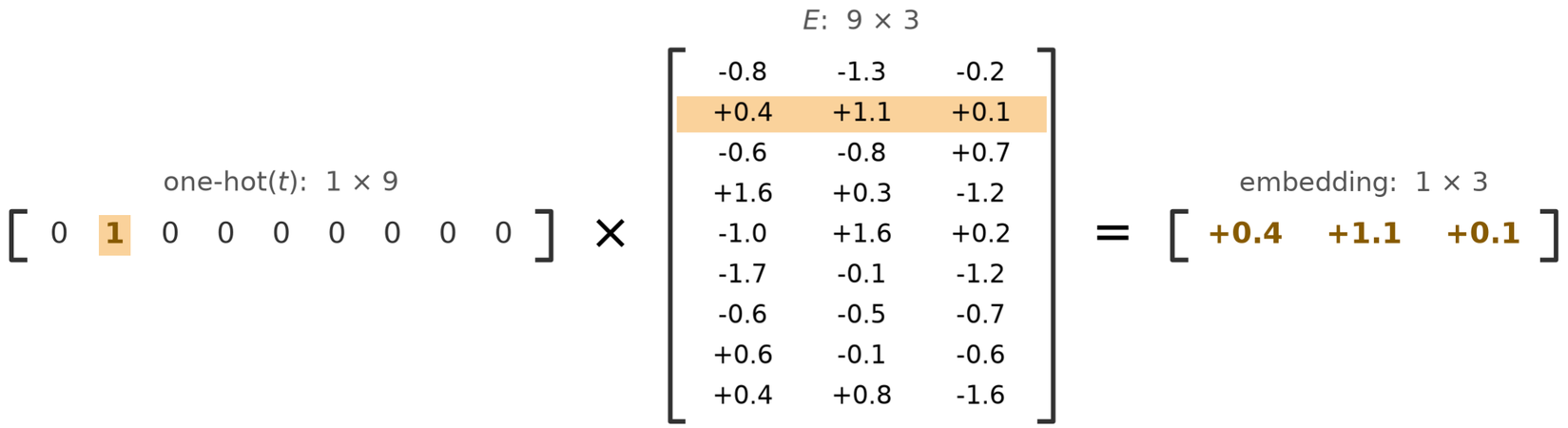
- Count how often each word (row) has each other word (column) within the window: each row is a word **in terms of its neighbours**
- A row is as long as the vocabulary. With thousands of words, it is almost all zeros
- An **embedding** plays that role in a short vector of $d$ numbers

# Where the vectors live: the embedding table




- Each token is one row of the **embedding table** $E$, a matrix of shape (vocabulary size, $d$), here $9 \times 3$. The highlighted row is “cat” (id 1)
- The numbers are parameters. They start random, and training moves them so each row describes its token by its neighbours

# Breakdown: a lookup is a matrix product




- The 1 picks out row 1 of $E$ and every other row is multiplied by 0, so the product **is** that row. The layer just looks the row up
- A long, mostly-zero vector in, a short dense one out: 9 numbers become 3. With 20,000 words and $d = 300$, 20,000 become 300

# The embedding layer in PyTorch

```python
import torch
from torch import nn

emb = nn.Embedding(num_embeddings=9, embedding_dim=3)       # the table E: 9 tokens, 3 dimensions
ids = torch.tensor([[ID[w] for w in SEQ]])                  # (batch, length) = (1, 12)
out = emb(ids)                                              # (batch, length, dim)
print(ids.shape, "->", out.shape)

onehot = nn.functional.one_hot(ids, 9).float()              # (1, 12, 9)
print(torch.allclose(onehot @ emb.weight, out))             # a lookup equals one-hot times the table
```

```
torch.Size([1, 12]) -> torch.Size([1, 12, 3])
True
```

- Input: ids of shape (batch, length). Output: one vector per id, shape (batch, length, $d$)
- `emb.weight` is the table, an ordinary parameter that backpropagation updates

# How do we train the table?

## Inside a task

- Put the table at the front of a model with a task, such as predicting the next word
- That task’s loss is the training signal
- The vectors are a by-product

## On neighbours alone

- Train on one question: which words appear near each other?
- word2vec’s own loss is the training signal
- The vectors are the product. This is the route we use

Either way: start from random rows, compute a loss, and backpropagation moves the rows of the table

# The skip-gram loss

One training example is a centre word with vector $\mathbf{u}$, one real neighbour $\mathbf{v}$, and a few random words $\mathbf{v_n}$.

$$\text{loss} = \underbrace{-\log \sigma(\mathbf{u} \cdot \mathbf{v})}_{\text{real neighbour}} \underbrace{-\sum_{n} \log \sigma(-\mathbf{u} \cdot \mathbf{v_n})}_{\text{random words}}$$

- It is word2vec's own loss. Training makes it small by scoring the real neighbour high and the random words low
- The same idea as the twin network's contrastive loss, in a different form: real pairs together, other pairs apart

# Breakdown: the skip-gram loss

| Piece | What it is |
|---|---|
| $\mathbf{u}$ | The **centre** word's vector: its row of the table |
| $\mathbf{v}$ | A **real neighbour**'s vector: a word inside the centre word's window |
| $\mathbf{v_n}$ | A **random** word's vector, drawn from the vocabulary. A few per example, so the sum |
| $\mathbf{u} \cdot \mathbf{v}$ | The score for the pair, read through $\sigma$ as a probability that they are neighbours |
| $-\log \sigma(\mathbf{u} \cdot \mathbf{v})$ | **Real-neighbour term**: small when the real pair scores high |
| $-\log \sigma(-\mathbf{u} \cdot \mathbf{v_n})$ | **Random-word term**: the minus sign inside flips it, so it is small when the random word scores **low** |

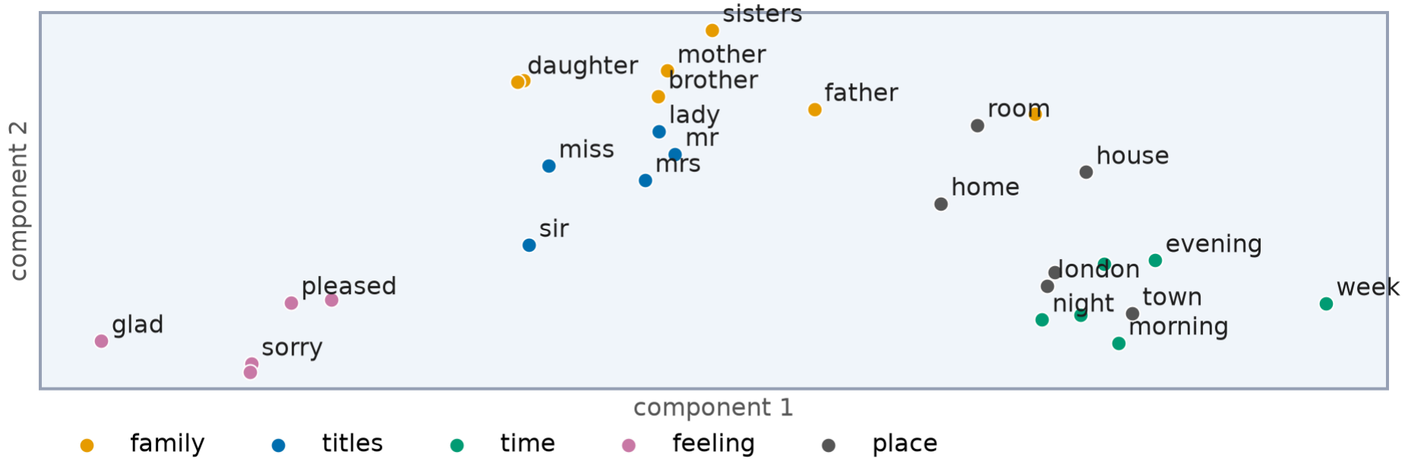
- Without the random-word term, every vector could point the same way and the real-neighbour term would be satisfied
- How it is used: for each (centre, neighbour) pair, raise the score of the two rows, and lower the centre's score with a few random words (**negative samples**). Backpropagation moves the rows

# The skip-gram loss with real numbers

| Pair | Dot product | The term with the number in | Loss |
|---|---|---|---|
| Real neighbour, score high | $+2$ | $-\log \sigma(2)$ | 0.13 |
| Real neighbour, score low | $-2$ | $-\log \sigma(-2)$ | 2.13 |
| Random word, score low | $-2$ | $-\log \sigma(2)$ | 0.13 |
| Random word, score high | $+2$ | $-\log \sigma(-2)$ | 2.13 |

- Because $\sigma(2) = 0.88$: a real pair that scores $+2$ costs 0.13, and one that scores $-2$ costs 2.13
- For a random word the signs flip. Scoring high costs 2.13, so training lowers that score

# What the vectors look like



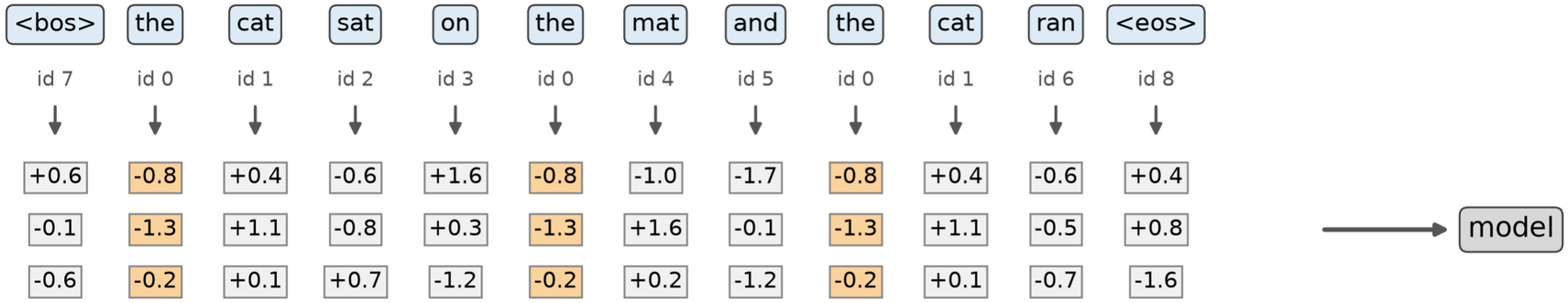

- word2vec (50 dimensions) trained on 128,565 words of *Pride and Prejudice*. Shown: a 2-D PCA of 29 chosen words, colored by group (only some labeled)
- Nobody told it the groups. Words used in similar company ended up near each other. Two dimensions keep only 28% of the variance, so it is a rough view

# Nearest neighbours

| Word | Its nearest neighbours |
|---|---|
| mother | father, sister, commendation, husband, song |
| london | town, meryton, scotland, hertfordshire, militia |
| mr | collins, miss, wickham, steward, darcy |
| morning | day, saturday, next, third, arrival |
| money | knows, exertion, respect, done, pardon |

- Nearest by cosine similarity of the vectors. Family words, places and titles come out sensible
- “Near” means “used in similar company”, not “means the same”. Rare words and a small book give some odd neighbours, such as for “money”

# A sentence becomes a sequence of vectors




- Look up each token's row: the sentence becomes a **sequence of vectors**, shape (12, 3). This is what the model is given
- The order is kept, and a repeated word gets the same vector: "the" (orange) three times
- A batch has shape (batch, length, $d$), as in the PyTorch slide

# One word, one vector

### “bank”

- “she sat on the river **bank**”
- “she went to the **bank** for a loan”

### One vector

- Every encoding so far gives “bank” the same vector in both sentences
- The vector cannot depend on the words around it

Making a word’s vector depend on its context is where the next lecture, and then attention, go

# Takeaway

**An embedding layer turns ids into vectors, and word2vec trains them so that words used in similar company end up close.**

# Summary

# Recap: ways to encode a token

| | Vector size | Similar words close? | Built by |
|---|---|---|---|
| One-hot | vocabulary size | No: all equally far | By hand |
| Bag of words | vocabulary size | No, and word order is lost | By counting |
| Embedding inside a task | $d$, small | Only as far as the task needs | Training on the task's loss |
| word2vec | $d$, small | Yes, for words used in similar company | Training on neighbours |

- All four give a word the **same** vector wherever it appears

# Recap: key ideas

- A **sequence** is data where order matters, and the basic task is to predict the next item. Items can be numbers, or symbols such as words, letters in DNA, pages, notes
- Symbols become **tokens**, then **ids**. Every choice (case, punctuation, characters or words) throws something away. Special tokens (<unk>, <pad>, <bos>, <eos>) cover the unknown, padding and the ends
- **One-hot** and **bag of words** are simple encodings with no sense of similarity or order
- An **embedding** is a learned vector per token. A lookup is a one-hot vector times a table. **word2vec** trains it with the skip-gram loss on neighbours

# Where this goes next

- A language model: use the vectors to predict the next word, first from a fixed window of words, and see where a window runs out
- A model that carries a summary of everything seen so far, and updates it with each new word: a recurrent network
- Then attention: a vector that depends on the words around it, so “bank” can differ
- Not covered here: how subwords are built, seq2seq, and pretrained embeddings